Clock Domain Crossing

Part 3

Amr Adel Mohammady

/amradelm

Save The Palestinian Children

Till Nov,2023

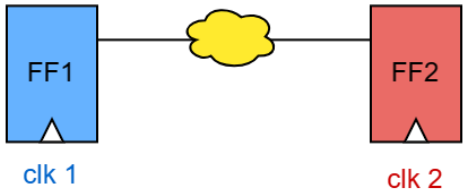
Israel has killed more than 13,000 children in Gaza since October 7 while others are suffering from severe malnutrition and do not “even have the energy to cry”, says the United Nations Children’s Fund (UNICEF).

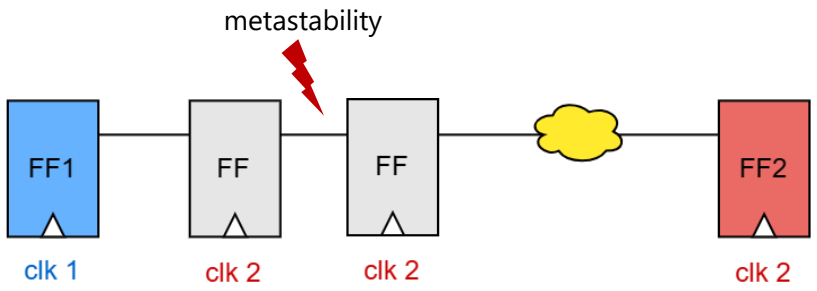
“Thousands more have been injured or we can’t even determine where they are. They may be stuck under rubble … **We haven’t seen that rate of death among children in almost any other conflict in the world**”

-**UNICEF** Executive Director

Introduction

- In the previous part we discussed CDC synchronizers their rules and how to ensure safe capture by launching a wide pulse.
- **Our CDC concerns till now are:**
  - Data corruption: Partially fixed. The system, till now, can only send 1-bit data. Also, this data has a varying arrival time.
  - Data incoherence: Fixed. The metastable value settles within the synchronizers at 0 or 1 and then propagates to all the 2nd domain blocks with the same settled value. We saw the divergence issues and learned how to avoid them.
  - Data loss: Fixed. The pulse is wide enough that it won't be missed by domain 2
  - Data duplication: Not fixed
  - Chip burning: Fixed. We limited the metastability propagation between the synchronizers and reduced its occurrence frequency.
  - In this part we will handle data duplication

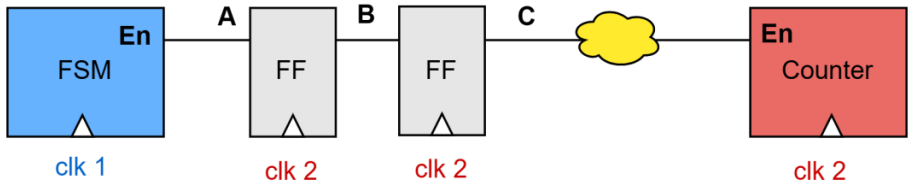


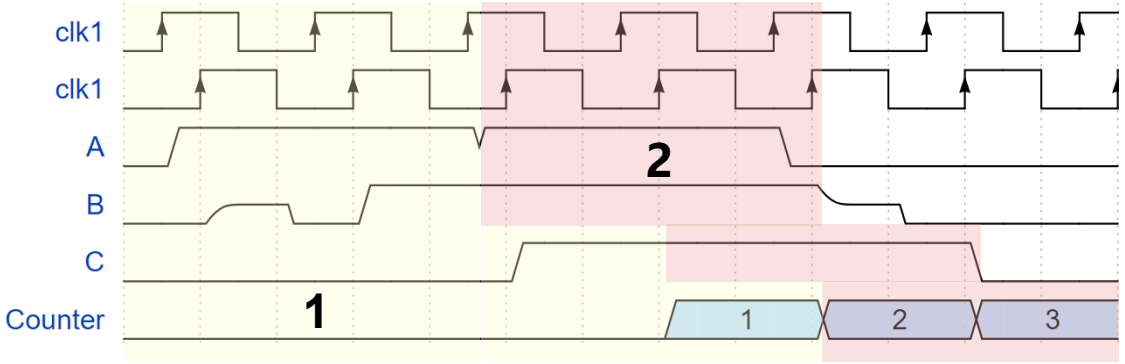


Data Duplication

Data Duplication

- **Consider the following example : Domain 1 sends a control signal to domain 2 to enable a counter. Each enable means one count up**
  1. Domain 1 sends a logic “1” to domain 2.
     1. The data goes through metastability and settles at “logic 0”.
     2. Then finally settles at 1 because the pulse was wide enough.
     3. It then safely reach C and cause the counter to increment one count.
  2. Domain 1 sends another logic “1” to domain 2.
     1. This time the data come out of metastability as “logic 1” which happens to be the correct data.
     2. The next cycle we get another “logic 1” because we made the pulse wide to ensure safe capture.
     3. The counter gets 2 enable signals causing it to count 2 times while it was intended to count only once.
- This example shows the problem with data duplication even if the two clocks have the same frequency.
- We need to find a way to make domain 2 sees a single cycle pulse for each wide pulse sent by domain 1



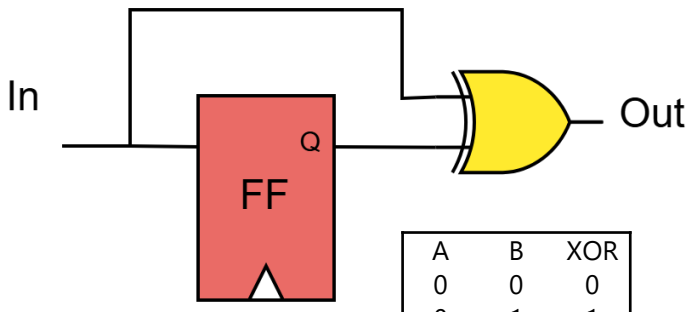


Pulse Synchronizer

Pulse Generator

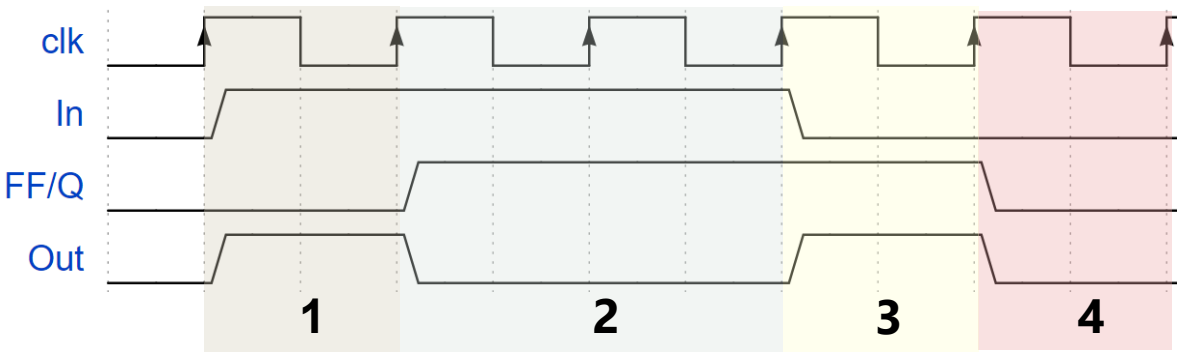
- **Consider the circuit on the right:**

1. The input **changes** to logic 1.
   The XOR inputs are (In= 1) and (FF/Q= 0) so the (output= 1)
2. The input remains at logic 1. The FF now stores the previous value "1".
   The XOR inputs are (In= 1) and (FF/Q= 1) so the (output= 0)
   The output remains "0" as long as the input remains constant.
3. The input **changes** to logic 0.
   The XOR inputs are (In= 0) and (FF/Q= 1) and so the (output= 1)
4. The input remains at logic 0. The FF now stores the previous value "0".
   The XOR inputs are (In= 0) and (FF/Q= 0) so the (output= 0)
   The output remains "0" as long as the input remains constant.



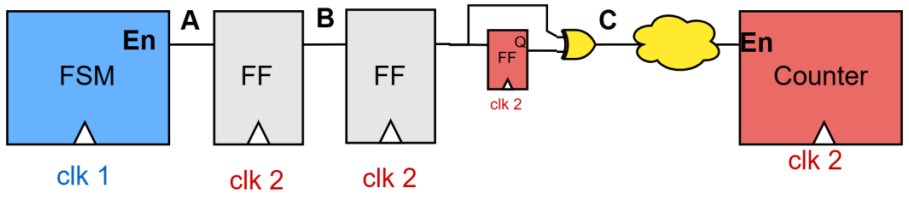
A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

- As we can see, this circuit outputs "1" for one cycle whenever the input **changes/toggles**. And outputs "0" as long as the input remain constant.
- This circuit is called a pulse generator and will help us deal with the CDC data duplication issue.



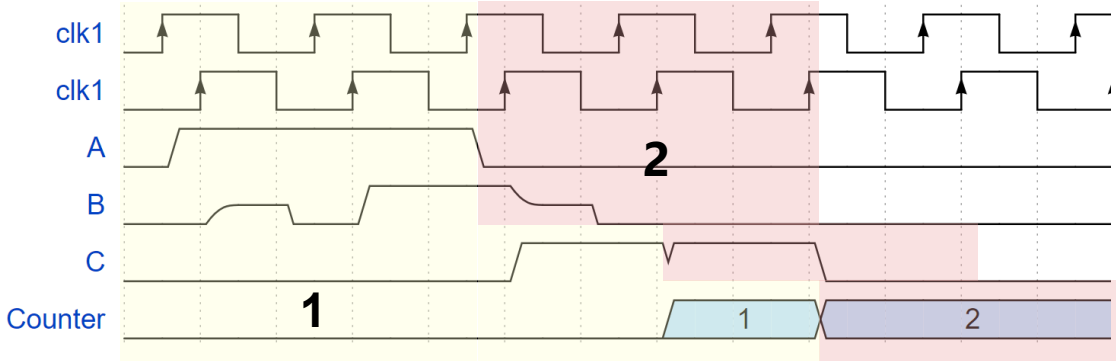
Data Duplication

- **Lets analyze the same example we saw earlier but now with the pulse generator added:**

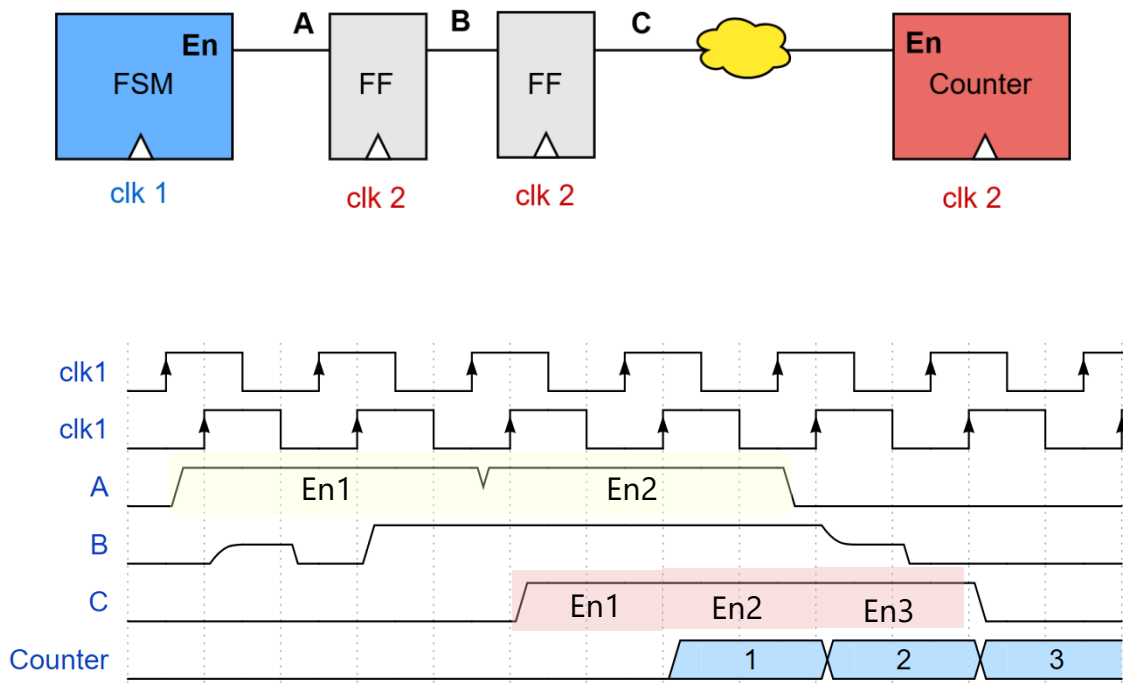


1. Domain 1 sends a logic “1” to domain 2.
    1. The data goes through metastability and settles at “logic 0”.
    2. Then finally settles at 1 because the pulse was wide enough.
    3. It then safely reaches the pulse generator. The pulse generator ensures that the pulse remains 1 for only one cycle as long as the input to remains constant
2. After some time domain 1 sends another enable to domain 2. The enable is sent by toggling the signal to “0”.
    1. The data goes through metastability and settles at “logic 0”.
    2. The next cycle we get another “logic 0” because we made the pulse wide to ensure safe capture.
    3. The pulse generator outputs “logic 1” for only one cycle.

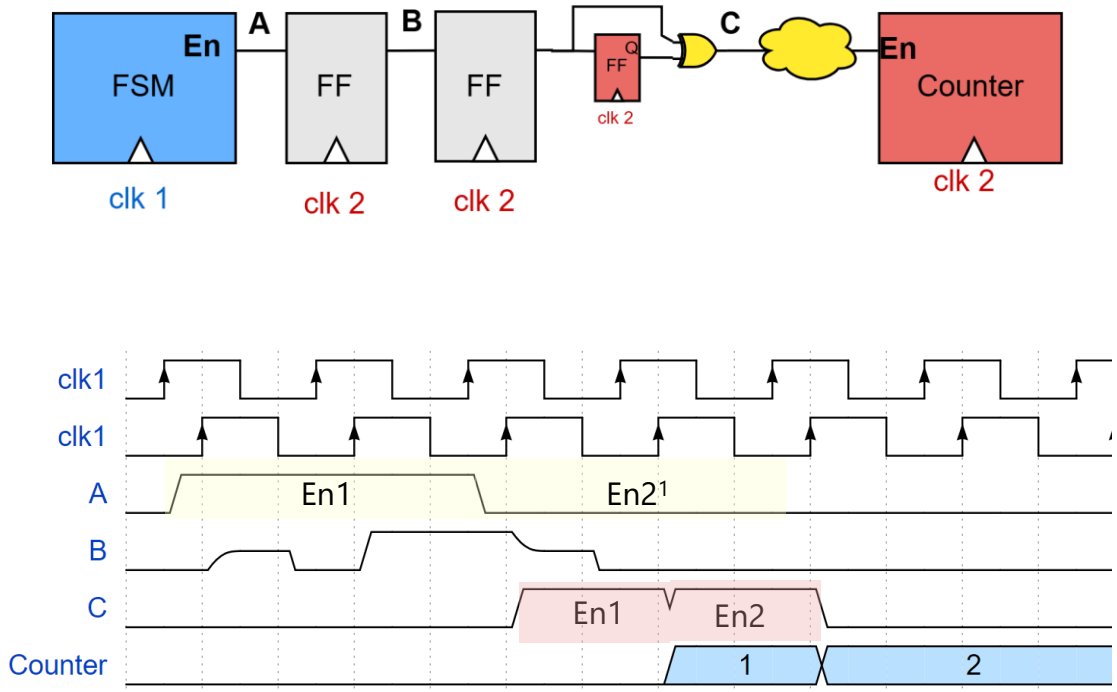
- As you can see the pulse generator fixed the data duplication issue because it converts a wide pulse into a single cycle pulse.



Data Duplication



Without Pulse generator



With Pulse generator

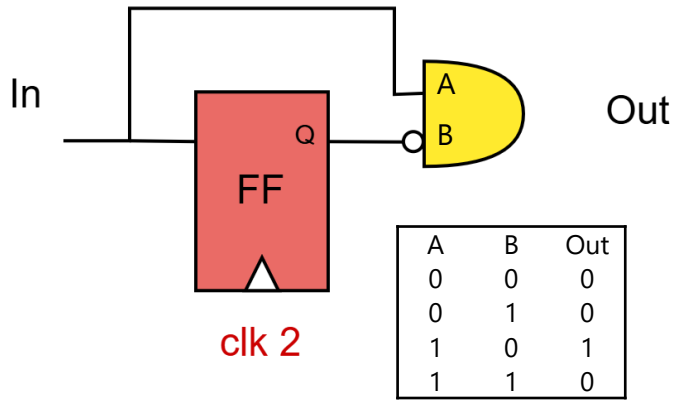
[1]: Notice how En2 in the pulse generator example is a toggle and not logic 1

Edge Synchronizers

Edge Detector

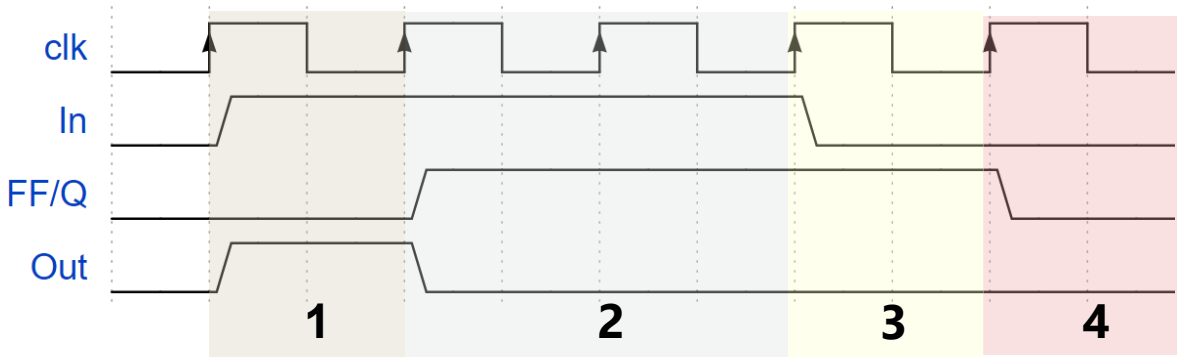
- **Consider the circuit on the right:**

1. The input changes to logic 1.
   The inputs to the AND are In: “1” and FF/Q: “0” so the output: “1”
2. The input remains at logic 1. The FF now stores the previous value “1”.
   The AND inputs are In: “1” and FF/Q: “1” so the output: “0”
   The output remains “0” as long as the input remains constant.
3. The input changes to logic 0.
   The inputs to the AND are In: “0” and FF/Q: “1” and so the output: “0”
4. The input remains at logic 0. The FF now stores the previous value “0”.
   The AND inputs are In: “0” and FF/Q: “0” so the output: “0”
   The output remains “0” as long as the input remains constant.

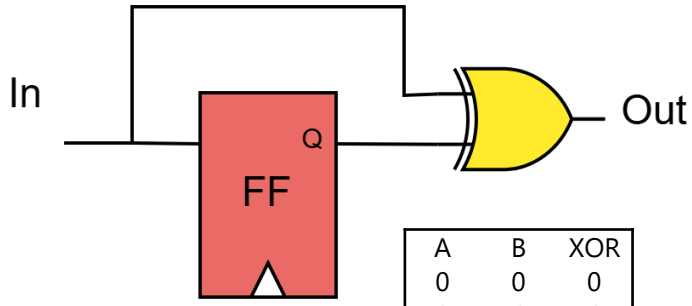


A	B	Out
0	0	0
0	1	0
1	0	1
1	1	0

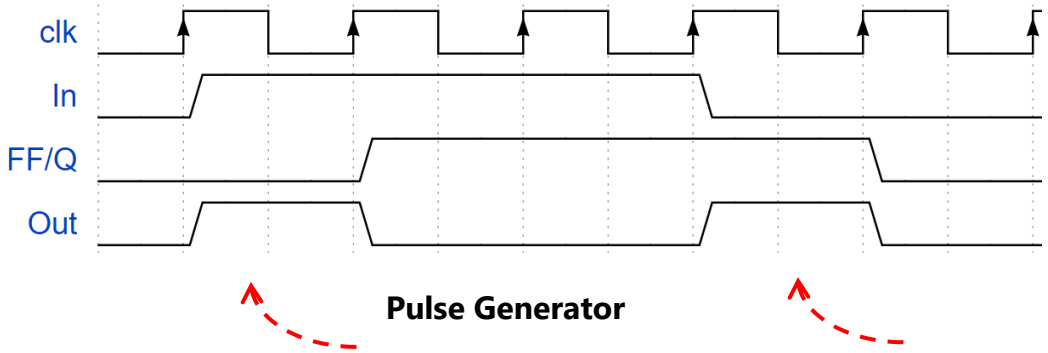
- Compared to the pulse generator, this circuit produced a pulse only when the input changed from 0 → 1 while the pulse generator produced a pulse whether the input changed from 0 → 1 or from 1 → 0.

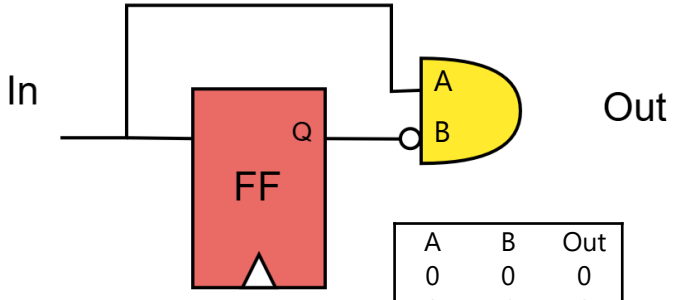


Pulse Generator vs Edge Detector

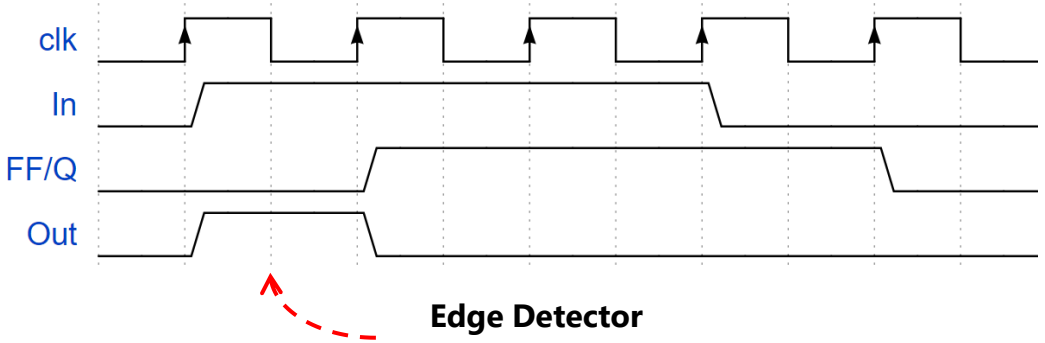


A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

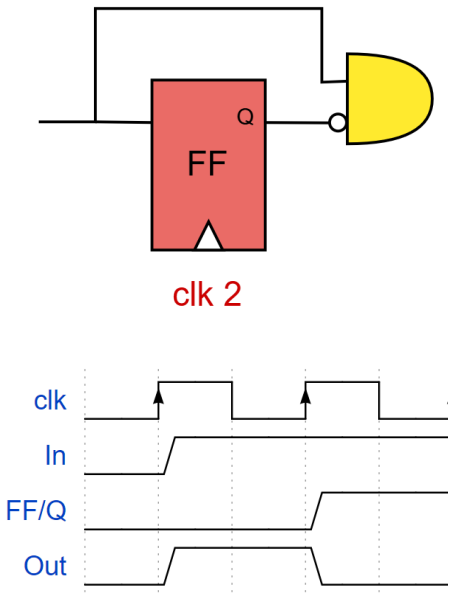




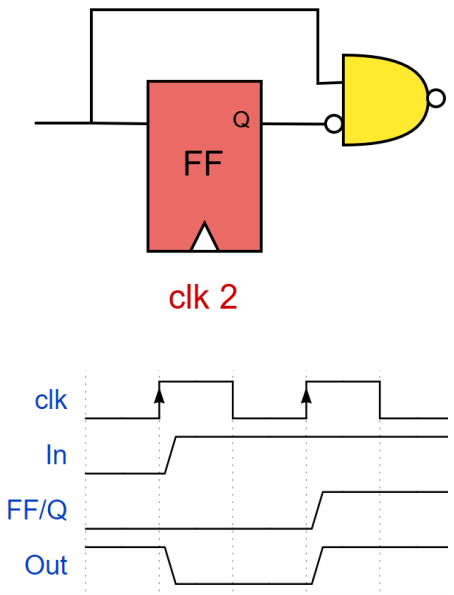
A	B	Out
0	0	0
0	1	0
1	0	1
1	1	0



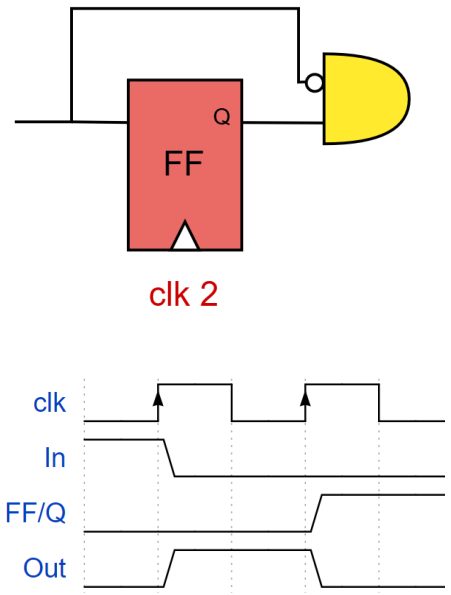
Edge Detector Types



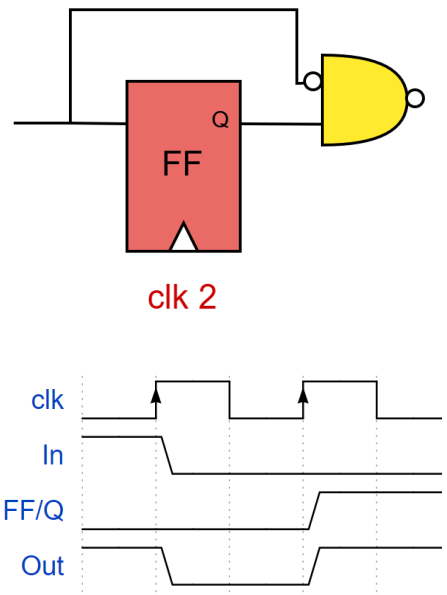
Rising Edge Detection
Active High Output



Rising Edge Detection
Active Low Output



Falling Edge Detection
Active High Output



Falling Edge Detection
Active Low Output

Conclusion

- **We want to know what we solved till now. Our CDC concerns were:**
  - Data corruption: Partially fixed. The system, till now, can only send 1-bit data. Also, this data has a varying arrival time.
  - Data incoherence: Fixed. The metastable value settles within the synchronizers at 0 or 1 and then propagates to all the 2nd domain blocks with the same settled value
  - Data loss: Fixed. The pulse is wide enough that it won't be missed by domain 2
  - Data duplication: Fixed
  - Chip burning: Fixed. We limited the metastability propagation between the synchronizers and reduced its occurrence frequency.
- In the next parts we will see how to send a multi-bit signal.

References

1) https://www.uio.no/studier/emner/matnat/ifi/IN3160/v21/timeplan/in3160-l92-clock-domains.pdf
2) https://ieeexplore.ieee.org/document/1676187
3) https://www.edn.com/keep-metastability-from-killing-your-digital-design/
4) https://people.ece.ubc.ca/~edc/7660.jan2018/lec11.pdf
5) https://www.onsemi.com/pub/Collateral/AN1504-D.PDF

Thank You!